NATIONAL UNIVERSITY OF MODERN LANGUAGE ISLAMABAD

Assignment:-03

Submitted to:

Muhammad Usman Shahid

Submitted by:

Zulqarnain Hassan

Roll No: …… SP21379
Subject: …… Software Re-Engineering
Department: …… BSSE-7-Afternoon
Date: …… 24-12-2026

Source Code Reverse Engineering of Trivia Legacy Code

1. Overview of the Legacy Code

The Trivia code is a procedural, state-driven Java program implementing a simple trivia game. It consists primarily of:

A main game class (Game / TriviaGame depending on version)

State variables:

players, places, purses

currentPlayer, isGettingOutOfPenaltyBox

Game logic methods:

add(String playerName)

roll(int roll)

askQuestion()

wasCorrectlyAnswered()

wrongAnswer()

The code heavily relies on conditional branching, loops, and shared mutable state, which makes it a good candidate for CFA, DFA, and slicing.

2. Control Flow Analysis (CFA)

Control Flow Analysis identifies all possible execution paths in the program.

2.1 Intra-Procedural Control Flow Analysis

Intra-procedural CFA examines control flow within a single method.

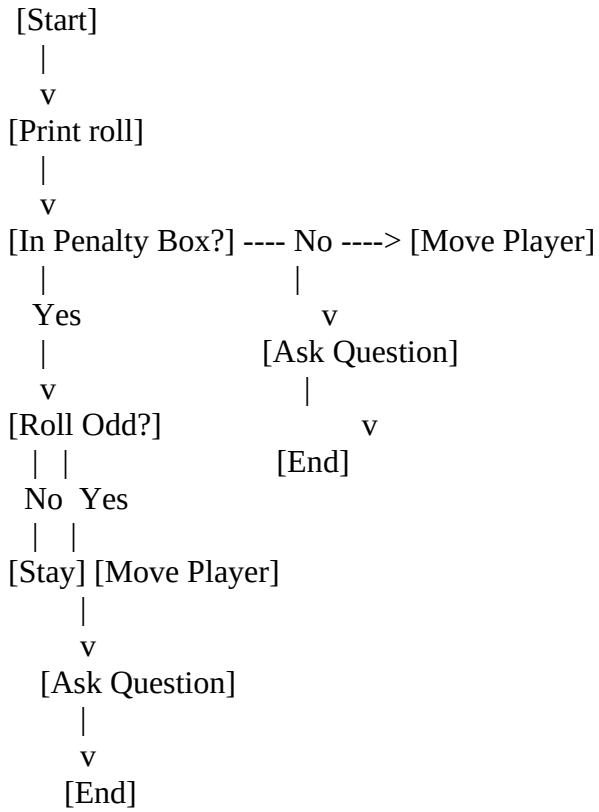
Example: roll(int roll) Method

Simplified Logic

```
roll(int roll):
  print roll
  if player is in penalty box:
    if roll is odd:
      isGettingOutOfPenaltyBox = true
      move player
      askQuestion
    else:
      isGettingOutOfPenaltyBox = false
```

```
  else:
     move player
     askQuestion
```

Intra-procedural CFG (Textual)



Explanation

The method has multiple decision nodes (if statements)

Execution paths diverge based on:

Player penalty status

Dice roll parity

This complexity increases testing paths and defect probability

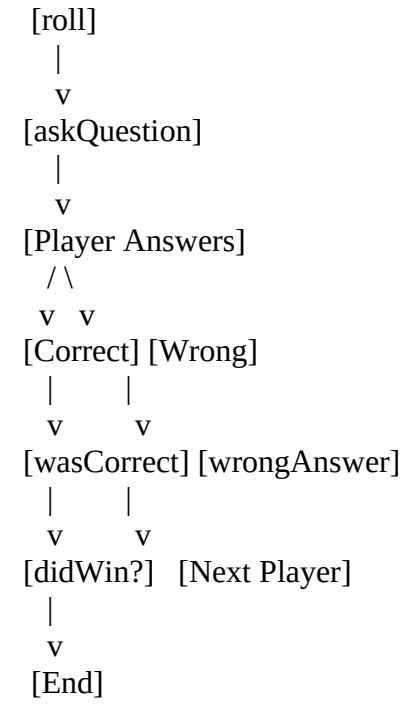
2.2 Inter-Procedural Control Flow Analysis

Inter-procedural CFA analyzes control flow across method boundaries.

Key Method Interactions

roll() → askQuestion()
wasCorrectlyAnswered() → didPlayerWin()
wrongAnswer() → roll()

Inter-Procedural CFG (Simplified)



Explanation

Control flows non-linearly across methods

Shared global state (currentPlayer, purses) creates hidden dependencies

Difficult to trace execution without CFG

3. Data Flow Analysis (DFA)

Data Flow Analysis tracks how data is defined, modified, and used.

3.1 Key Data Elements

Variable	Type	Purpose
currentPlayer	int	Tracks active player
places[]	int[]	Player positions
purses[]	int[]	Player scores
isGettingOutOfPenaltyBox	boolean	Penalty logic flag

3.2 Definition–Use Chains (DU Chains)

Example: currentPlayer

Definitions: - Initialized at game start - Modified in nextPlayer() logic

Uses: - roll() – movement - askQuestion() – category - wasCorrectlyAnswered() – scoring

Data Flow Graph (Textual)

```
[currentPlayer = 0]
        |
        v
     roll()
        |
        v
 askQuestion()
        |
        v
wasCorrectlyAnswered()
        |
        v
 currentPlayer++
```

Observations

Many variables are globally mutable

High coupling between methods

No encapsulation of player state

4. Program Slicing

Program slicing extracts only the statements affecting a particular variable or behavior.

4.1 Backward Program Slicing

Slice Criterion

Variable: purses[currentPlayer] at wasCorrectlyAnswered()

Backward Slice Code

```
boolean wasCorrectlyAnswered() {
   if (isGettingOutOfPenaltyBox) {
      purses[currentPlayer]++;
      boolean winner = didPlayerWin();
      currentPlayer++;
      if (currentPlayer == players.size()) currentPlayer = 0;
      return winner;
   }
   return true;
}
```

Explanation

Includes all statements that affect score increment

Removes unrelated logic (questions, categories)

4.2 Forward Program Slicing

Slice Criterion

Statement: roll(int roll)

Forward Slice Code

```
void roll(int roll) {
   if (inPenaltyBox[currentPlayer]) {
      if (roll % 2 != 0) {
         isGettingOutOfPenaltyBox = true;
         places[currentPlayer] += roll;
         askQuestion();
      }
   } else {
      places[currentPlayer] += roll;
      askQuestion();
   }
}
```

Explanation

Shows all effects of rolling the dice

Excludes scoring and win logic

5. Key Reverse Engineering Insights

CFA exposed high cyclomatic complexity

DFA revealed excessive shared state

Program slicing showed logic tangling

Re-engineering Recommendations

Encapsulate player state into a Player class

Reduce global variables

Replace conditionals with polymorphism

Separate game rules from I/O